

线性表: $n (≥0)$ 个表项的有限序列: $L = (a_1, a_2, \dots, a_n)$

$\uparrow$
 表头 head 无前驱

$\uparrow$
 表尾 tail 无后继

操作: 创建, 销毁, 长度, 输出, 判空, 增, 删, 改, 查 {按位置
按数据

① 顺序表: 用一维数组实现

内存:

0

1

...

i

...

$\text{地址 } a$
 $\text{地址 } loc(i) = loc(i-1) + l = a + i \cdot l$

$(l: \text{数据类型所占内存大小})$

{ 顺序访问
随机访问 (i) }

内存空间的开辟 {静态
动态 (各种实现自己写一遍)}

② 单链表: Node:

element

link

内存:

1

3

2

4

1

 (Last)

可以在单链表最前面加上一个“附加头结点”, 其指向表的第一个元素, data 可不设 (或设为表长)

这样定位到 current 结点后, 插入/删除无需讨论是否为表头

$\begin{cases} \text{newNode} \rightarrow \text{link} = \text{current} \rightarrow \text{link}; \\ \text{current} \rightarrow \text{link} = \text{newNode}; \end{cases}$

$\begin{cases} \text{del} = \text{current} \rightarrow \text{link}; \\ \text{current} \rightarrow \text{link} = \text{del} \rightarrow \text{link}; \\ \text{delete del}; \end{cases}$

③ 循环链表: 表尾结点的 next 不是 NULL, 而是 first 结点或附加头结点

(需要同时知道 first 和 last, 遍历元素时以回到 first 为准)

④ 双向链表: Node:

link

data

rlink

 (即: 每个结点有前驱和后继)

一般使用附加头结点的双向循环链表:

* 链表应用: 多项式及其运算

• n 阶多项式 $P_n(x) = \sum_{i=0}^n a_i x^i = a_0 + a_1 x + \dots + a_n x^n$

用静/动态数组可以存放 a_i , 但对于大多系数为 0 的多项式 (eg. $3-5x^9+x^{100}$) 不经济

因此可用链表 (大小不必确定、方便运算时增删) Node:

系数

指数

|

 $\rightarrow \text{next}$

多项式加法即为两个链表的合并 (易写, 设两个检测指针按逻辑运算即可)

* 静态链表: Node[size] 数组, 其中 Node:

data

link

, 即用数组实现链表

eg:

	0	1	2	...	n
data		25	92	...	49
link	1	7	3	...	2